Mini Project Report

On

Online Examination Result System

(Stored procedures, functions and triggers in MySQL)

SUBMITTED BY:
Puneet Gupta
UID: 26MCI10246
CLASS: MCA (AI & ML)
SECTION: 24MAM4-4

SUBMITTED TO:
Mr. Mandeep Singh
Assistant Professor

Table of Content

S. No.	Name	Page no.
1	Introduction	4
2	Background	4
3	Objectives	4
4	Technologies Used	5
5	Requirements	5
6	System Design	5
7	Source Code & Output	7
8	Functionalities	14
9	Conclusion	14
10	Future Work	15
11	Learning Outcomes	15
12	GitHub Link	15
13	Bibliography	15

Acknowledgement

I would like to express my sincere gratitude to my guide, Mr. Mandeep Singh, for his guidance during this mini project. His suggestions on where the logic of a database application should live, and his insistence that every claim in the report be demonstrated rather than described, shaped the way this project was built.

I am also thankful to the Department of Computer Applications, Chandigarh University, for the laboratory and the resources that made this work possible, and to the faculty of the MCA (AI & ML) programme for the foundations in database systems on which the project rests.

I would like to thank my classmates who used the application while it was being built and reported the mistakes they found. Testing the batch entry from the screen rather than from the database console exposed an error that would otherwise have gone unnoticed.

Finally, I would like to thank my family for their patience and support during the period of this work.

Puneet Gupta
(26MCI10246)

1. Introduction

This report describes an online examination result system. It stores the marks a student scores in each exam, works out the percentage, assigns a letter grade, decides whether the student has passed, and ranks the students of a class. The system is a small web application written in Python, but the web pages are not the interesting part of it. Almost all of the work is done by the MySQL database underneath them.

The application is deliberately thin. When a teacher types a mark into a web page, the page does not calculate the percentage and it does not decide the grade. It sends the mark to a stored procedure and displays whatever the database sends back. If a mark is larger than the maximum for that paper, the page does not stop it either: the database refuses to store the row and returns the reason, which the page then shows to the user. The rule exists in one place, and every route to the data passes through it.

The project therefore consists of four tables, five views, five functions, seven stored procedures and five triggers. It is delivered with a demonstration dataset of eight students, four exams and twenty-three results, and with one page that deliberately breaks the rules so that the database's refusals can be seen rather than taken on trust.

2. Background

In a typical project of this kind, the marks are entered on a web page and the page itself works out the percentage and the grade. The code is short and it works, until the same data has to be entered from somewhere else. A second program, a spreadsheet import, or a correction typed directly into the database console knows nothing about the rule, and the table slowly fills with rows whose percentage does not match their marks. The same problem appears in the other direction: if the grade boundaries change, they have to be changed in every program that was written against the table.

A relational database can hold the rule itself. A trigger fires whenever a row is inserted, changed or deleted, so a calculation happens whatever the source of the change is. A stored procedure gives one entry point for an action that touches several tables, and keeps that action inside one transaction. A function returns a single value and can be called from a trigger, from a view or from an ordinary query, so a rule such as the grade boundaries is written once and used everywhere. A view that uses a window function gives a ranking without storing it, so a rank can never become stale. This project is built around those four ideas.

3. Objectives

- Move the marking rules out of the web page and into the database, so that they hold for every client and not only for one program.
- Calculate the percentage, the letter grade and the pass or fail flag automatically when a result row is written.
- Refuse a result that breaks a rule, and return a message that a person can read instead of a crash.
- Enter the marks of a whole class in a single call, skipping the rows that are not valid instead of stopping at the first one.
- Calculate ranks when they are read rather than storing them, so that a rank always agrees with the marks.
- Record every insert, update and delete in an audit trail that is written by a trigger and not by the application.

- Show all of this on a website, so that the behaviour of the database can be seen and not only described.

4. Technologies Used

- **Database:** MySQL 8.0, developed and tested on MariaDB 11.8, which is compatible with it. Holds the tables, the routines and the data.
- **Language:** SQL for the queries and the views, and the MySQL stored program language for the functions, procedures and triggers.
- **Constraints:** Primary keys, foreign keys, a unique key on the pair of student and exam, and a check constraint on negative marks. These hold the simple rules; the rest are in code.
- **Functions:** Five stored functions. They return one value and are called from triggers, views and ordinary queries alike.
- **Triggers:** Five triggers. A before-insert and a before-update trigger copy of the marking logic, and three audit triggers, one for each of insert, update and delete.
- **Procedures:** Seven stored procedures. They use explicit cursors, OUT parameters, SIGNAL to raise an error, and handlers to recover from one.
- **Web Framework:** Python 3 and Flask. One route per action, each calling one routine and rendering one template.
- **Database Driver:** PyMySQL, which returns each row as a dictionary so the templates can read columns by name.
- **Front End:** HTML and CSS rendered through Jinja2 templates. No external library and no JavaScript framework; the styling is inline in a single base template.
- **Client Tools:** The mysql command-line client for building the schema and running the demonstration script, and a web browser for the application.

5. Requirements

- MySQL 8.0 or MariaDB 10.5 or later, running locally or on a server.
- Python 3.9 or later with pip.
- Flask 3.x and PyMySQL 2.x, both listed in requirements.txt.
- A database account that is allowed to create routines, used once to build the schema; the web application itself uses a smaller account.
- About 20 MB of disk space, and any modern web browser.

6. System Design

Architecture Overview

The system is a small three-tier application. The presentation tier is the browser, which receives finished HTML pages. The application tier is a Flask process that accepts a form, calls one routine in the database, and hands the rows that come back to a template. The data tier is MySQL, and it is where the rules live. The three tiers talk in one direction only: the browser asks the application, the application asks the database, and the answer travels back. The application tier holds no state about a result; it keeps no copy of the percentage and does not remember which grades exist.

Component Overview

Ø **Frontend:** The browser receives plain HTML built from Jinja2 templates. There is no client-side framework and no external stylesheet. The pages are the dashboard, the merit list for one

exam, the analytics page, the audit log, a what-if calculator, one student's report, a printable mark sheet, and a page that proves the rules.

- Ø **Backend:** The Flask application has one route per action. Each route collects the form values, calls a single stored procedure or reads a single view, and passes the rows to a template. It does not calculate a percentage, decide a grade or work out a rank. Where the database refuses an operation, the route shows the database's own message.
- Ø **Database:** MySQL owns every rule. Four tables hold the data, five functions return single values, five triggers enforce and record, seven procedures perform the actions, and five views present the results that the pages need.

Database Design

- **Tables (4):** students, exams, results and result_audit. The results table carries the foreign keys, the unique key on the pair of student and exam, and the check that marks are not negative.
- **Functions (5):** fn_calc_percentage, fn_get_grade, fn_get_rank, fn_grade_points and fn_class_topper.
- **Procedures (7):** sp_generate_result, sp_generate_exam_results, sp_merit_list, sp_student_report, sp_class_statistics, sp_update_marks and sp_delete_result.
- **Triggers (5):** trg_results_bi and trg_results_bu calculate the derived values, and trg_results_ai, trg_results_au and trg_results_ad write the audit trail.
- **Views (5):** v_result_summary, v_exam_summary, v_grade_distribution, v_student_totals and v_subject_summary.
- **Grade scale:** 90 and above is A+, 80 is A, 70 is B+, 60 is B, 50 is C, 40 is D, and anything below 40 is F. A student passes at 40 per cent. The scale is written once, in fn_get_grade.

Data Flow

The figure below follows one result from the teacher's form to the page that displays it. Each step names the object that holds the rule, so the diagram can be read as a map of the schema. The teacher supplies only a student and a mark. The procedure writes the row, the before-insert trigger fills the derived columns and rejects a mark above the maximum, and the audit trigger records what happened. The views and procedures that follow never change the data; they read it and present it. Because the ranks and the statistics are calculated at read time, correcting a mark needs no repair of any stored total.

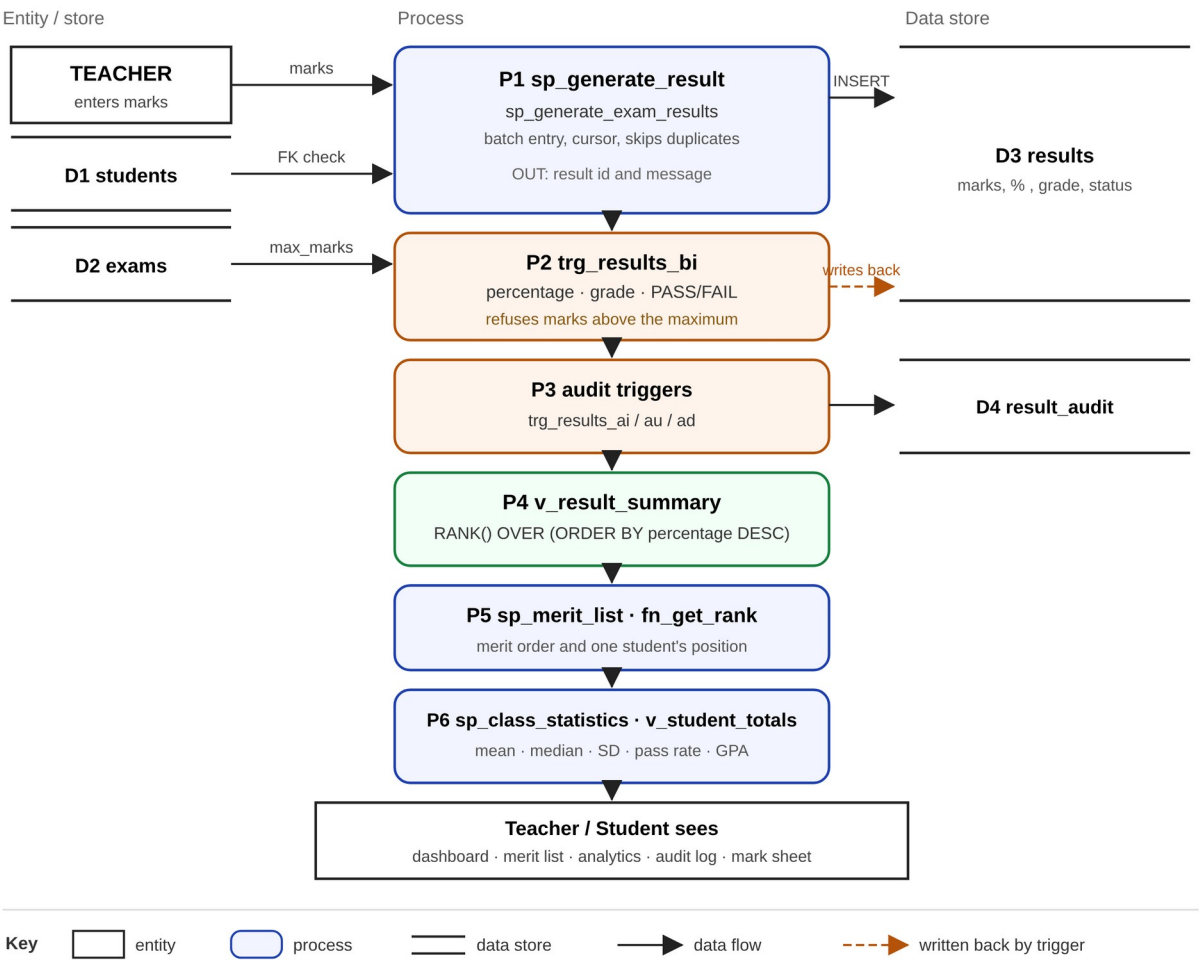


Fig. 1. Data flow through the system, with the database object that holds each rule.

7. Source Code & Output

The listings below are taken from the project files. They are printed as they are, including the $$ delimiter that the mysql client uses while a routine is being defined.

7.1 Tables and Constraints

The results table holds only what a person supplies and what the database derives. The percentage, the grade and the status columns are allowed to be empty at the moment of the insert, because the trigger fills them before the row reaches the disk.

Listing 7.1 The results table

```
CREATE TABLE results (
    result_id      INT AUTO_INCREMENT PRIMARY KEY,
    student_id     INT          NOT NULL,
    exam_id        INT          NOT NULL,
    marks_obtained DECIMAL(6,2) NOT NULL,
    percentage     DECIMAL(5,2) DEFAULT NULL,
    grade          VARCHAR(2)   DEFAULT NULL,
    result_status  VARCHAR(10)  DEFAULT NULL,
    entered_by     VARCHAR(50)  DEFAULT 'system',
    created_at     TIMESTAMP    DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_result_student FOREIGN KEY (student_id)
        REFERENCES students(student_id) ON DELETE CASCADE,
    CONSTRAINT fk_result_exam FOREIGN KEY (exam_id)
        REFERENCES exams(exam_id) ON DELETE CASCADE,
    CONSTRAINT uq_result_once UNIQUE (student_id, exam_id),
    CONSTRAINT chk_marks_positive CHECK (marks_obtained >= 0)
) ENGINE=InnoDB;

CREATE INDEX idx_results_exam    ON results (exam_id);
```

```
CREATE INDEX idx_results_student ON results (student_id);
CREATE INDEX idx_results_pct     ON results (exam_id, percentage);
```

7.2 The Functions

The percentage is clamped before it is stored, in a wide data type, so that an absurd input is reduced to 100 instead of overflowing the column.

Listing 7.2 fn_calc_percentage

```
CREATE FUNCTION fn_calc_percentage(p_marks DECIMAL(6,2), p_max INT)
RETURNS DECIMAL(5,2)
DETERMINISTIC
BEGIN
   -- The intermediate value is held in a wide DECIMAL so that an absurd
   -- input (999 out of 75 = 1332%) is clamped instead of overflowing.
   DECLARE v_raw DECIMAL(12,2);
   DECLARE v_pct DECIMAL(5,2);

   IF p_max IS NULL OR p_max = 0 THEN
      RETURN 0.00;
   END IF;

   SET v_raw = ROUND((p_marks * 100.0) / p_max, 2);

   IF v_raw < 0   THEN SET v_raw = 0;   END IF;
   IF v_raw > 100 THEN SET v_raw = 100; END IF;

   SET v_pct = v_raw;
   RETURN v_pct;
END $$
```

Listing 7.3 fn_get_grade

```
CREATE FUNCTION fn_get_grade(p_pct DECIMAL(5,2))
RETURNS VARCHAR(2)
DETERMINISTIC
BEGIN
   RETURN CASE
      WHEN p_pct >= 90 THEN 'A+'
      WHEN p_pct >= 80 THEN 'A'
      WHEN p_pct >= 70 THEN 'B+'
      WHEN p_pct >= 60 THEN 'B'
      WHEN p_pct >= 50 THEN 'C'
      WHEN p_pct >= 40 THEN 'D'
      ELSE 'F'
   END;
END $$
```

7.3 The Trigger That Marks the Result

This is the centre of the project. The trigger reads the maximum for the paper, refuses a mark above it, and then computes the three derived columns. Notice that the grade is not computed here; the trigger calls fn_get_grade, so the boundaries remain in one place.

Listing 7.4 trg_results_bi

```
CREATE TRIGGER trg_results_bi
BEFORE INSERT ON results
FOR EACH ROW
BEGIN
   DECLARE v_max INT;

   SELECT max_marks INTO v_max FROM exams WHERE exam_id = NEW.exam_id;

   IF v_max IS NULL THEN
      SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Result rejected: that exam does not exist.';
   END IF;

   IF NEW.marks_obtained > v_max THEN
      SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Result rejected: marks cannot exceed the maximum for this paper.';
   END IF;

   SET NEW.percentage    = fn_calc_percentage(NEW.marks_obtained, v_max);
   SET NEW.grade         = fn_get_grade(NEW.percentage);
   SET NEW.result_status = IF(NEW.percentage >= 40, 'PASS', 'FAIL');
END $$

-- ------------------------------------------------------------------------
```

```
-- trg_results_bu : BEFORE UPDATE
--   Any change to the marks recomputes the percentage and the grade.
-- ------------------------------------------------------------------------
DROP TRIGGER IF EXISTS trg_results_bu $$
CREATE TRIGGER trg_results_bu
BEFORE UPDATE ON results
FOR EACH ROW
BEGIN
   DECLARE v_max INT;

   SELECT max_marks INTO v_max FROM exams WHERE exam_id = NEW.exam_id;

   IF NEW.marks_obtained > v_max THEN
      SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Update rejected: marks cannot exceed the maximum for this paper.';
   END IF;

   SET NEW.percentage    = fn_calc_percentage(NEW.marks_obtained, v_max);
   SET NEW.grade         = fn_get_grade(NEW.percentage);
   SET NEW.result_status = IF(NEW.percentage >= 40, 'PASS', 'FAIL');
END $$

-- ------------------------------------------------------------------------
-- trg_results_audit : AFTER INSERT / UPDATE / DELETE
-- ------------------------------------------------------------------------
DROP TRIGGER IF EXISTS trg_results_ai $$
CREATE TRIGGER trg_results_ai
AFTER INSERT ON results
FOR EACH ROW
BEGIN
   INSERT INTO result_audit (result_id, action, old_marks, new_marks, changed_by)
   VALUES (NEW.result_id, 'INSERT', NULL, NEW.marks_obtained, NEW.entered_by);
END $$

DROP TRIGGER IF EXISTS trg_results_au $$
CREATE TRIGGER trg_results_au
AFTER UPDATE ON results
FOR EACH ROW
BEGIN
   INSERT INTO result_audit (result_id, action, old_marks, new_marks, changed_by)
   VALUES (NEW.result_id, 'UPDATE', OLD.marks_obtained, NEW.marks_obtained, NEW.entered_by);
END $$

DROP TRIGGER IF EXISTS trg_results_ad $$
CREATE TRIGGER trg_results_ad
AFTER DELETE ON results
FOR EACH ROW
BEGIN
   INSERT INTO result_audit (result_id, action, old_marks, new_marks, changed_by)
   VALUES (OLD.result_id, 'DELETE', OLD.marks_obtained, NULL, OLD.entered_by);
END $$
```

7.4 Result Generation

The procedure is the single entry point for entering one result. It checks for a duplicate first, and it uses an exception handler to turn any error raised underneath, including the trigger's SIGNAL, into a message that the web page can display.

Listing 7.5 sp_generate_result

```
CREATE PROCEDURE sp_generate_result(
   IN  p_student_id INT,
   IN  p_exam_id    INT,
   IN  p_marks      DECIMAL(6,2),
   IN  p_entered_by VARCHAR(50),
   OUT p_result_id  INT,
   OUT p_message    VARCHAR(200)
)
BEGIN
   DECLARE v_exists INT DEFAULT 0;

   DECLARE EXIT HANDLER FOR SQLEXCEPTION
   BEGIN
      GET DIAGNOSTICS CONDITION 1 @errmsg = MESSAGE_TEXT;
      SET p_result_id = NULL;
      SET p_message = CONCAT('Rejected: ', @errmsg);
   END;

   SELECT COUNT(*) INTO v_exists
     FROM results WHERE student_id = p_student_id AND exam_id = p_exam_id;
```

```
    IF v_exists > 0 THEN
        SET p_result_id = NULL;
        SET p_message = 'A result already exists for this student in this exam.';
    ELSE
        INSERT INTO results (student_id, exam_id, marks_obtained, entered_by)
        VALUES (p_student_id, p_exam_id, p_marks, p_entered_by);

        SET p_result_id = LAST_INSERT_ID();
        SET p_message = 'Result generated successfully.';
    END IF;
END $$
```

7.5 Batch Entry With a Cursor

The batch procedure receives a comma-separated list of student and marks pairs. A derived table of the numbers one to twenty splits the string: SUBSTRING_INDEX selects the first n pieces, and a second SUBSTRING_INDEX with a negative count keeps only the last of those, which is the nth piece itself. The cursor then walks the pieces. A continue handler attached to the insert means that one bad row is counted and skipped rather than aborting the batch.

Listing 7.6 The loop that inserts each row and counts what was skipped

```
OPEN cur;
    read_loop: LOOP
        FETCH cur INTO v_entry;
        IF v_done THEN
            LEAVE read_loop;
        END IF;

        SET v_pos = LOCATE(':', v_entry);
        IF v_pos > 0 THEN
            SET v_sid   = CAST(SUBSTRING(v_entry, 1, v_pos - 1) AS UNSIGNED);
            SET v_marks = CAST(SUBSTRING(v_entry, v_pos + 1) AS DECIMAL(6,2));

            BEGIN
                DECLARE CONTINUE HANDLER FOR SQLEXCEPTION
                BEGIN
                    SET p_skipped = p_skipped + 1;
                END;

                INSERT INTO results (student_id, exam_id, marks_obtained, entered_by)
                VALUES (v_sid, p_exam_id, v_marks, p_entered_by);
                SET p_inserted = p_inserted + 1;
            END;
        ELSE
            SET p_skipped = p_skipped + 1;
        END IF;

    END LOOP;
```

7.6 Rank Calculation

The rank is not a column. It is produced by a window function inside a view, so every read sees a rank that agrees with the marks as they are at that moment. Students on the same percentage share a rank.

Listing 7.7 v_result_summary

```
CREATE OR REPLACE VIEW v_result_summary AS
SELECT r.result_id,
       s.student_id,
       s.roll_no,
       s.full_name,
       e.exam_id,
       e.exam_name,
       e.subject,
       e.max_marks,
       r.marks_obtained,
       r.percentage,
       r.grade,
       r.result_status,
       RANK() OVER (PARTITION BY r.exam_id ORDER BY r.percentage DESC) AS rank_no,
       r.created_at
  FROM results  r
  JOIN students s ON s.student_id = r.student_id
  JOIN exams    e ON e.exam_id    = r.exam_id;
```

7.7 Grade Point Average

The GPA is the average of the grade points of the student's exams on a ten-point scale. It is computed by aggregating fn_grade_points over the letter grades, so it follows the grade scale automatically. The overall percentage is the total marks divided by the total maximum, and the overall rank is a second window function over that percentage. The view v_student_totals holds all three, and the leaderboard on the dashboard reads it directly.

7.8 The Web Layer

The route below is the whole of the server side for entering one result. It validates that the form is not empty and that the mark is not negative, and then it hands everything to the procedure. There is no percentage, no grade and no comparison with the paper's maximum in this code, because none of those belong here.

Listing 7.8 The Flask route for one result

```
@app.route("/enter", methods=["POST"])
def enter_marks():
    """Enter one student's marks by calling sp_generate_result."""
    student_id = request.form.get("student_id", type=int)
    exam_id = request.form.get("exam_id", type=int)
    marks = request.form.get("marks", type=float)
    who = request.form.get("entered_by", "teacher") or "teacher"

    if not student_id or not exam_id or marks is None:
        flash("Please choose a student, an exam and enter the marks.", "error")
        return redirect(request.referrer or url_for("dashboard"))

    if marks < 0:
        flash("Marks cannot be negative.", "error")
        return redirect(request.referrer or url_for("dashboard"))

    try:
        _, out = call_proc("sp_generate_result",
                           [student_id, exam_id, marks, who],
                           outs=("rid", "msg"))
    except pymysql.MySQLError as e:
        flash(f"The database refused the entry: {db_error(e)}", "error")
```

Output

The outputs below are captured from a run of 03_demo.sql against the database, which exercises the routines without the web page in the way. The command was mysql -u root --force < 03_demo.sql; the --force flag allows the script to continue past the four statements that are expected to fail.

Output 7.1 Objects in the schema, and the functions called directly

```
+--------------+--------------+----------+----------+----------+--------------+----------+
| pct_68_of_100 | pct_52_of_75 | grade_92 | grade_87 | grade_33 | points_Aplus | points_F |
+--------------+--------------+----------+----------+----------+--------------+----------+
|        68.00 |        69.33 | A+       | A        | F        |         10.0 |      0.0 |
+--------------+--------------+----------+----------+----------+--------------+----------+
+-----------------------------------------------------+
| step                                                |
```

Output 7.2 The trigger marks the row, and four bad writes are refused

```
+------------+----------------+------------+-------+--------------+
| roll_no    | marks_obtained | percentage | grade | result_status |
+------------+----------------+------------+-------+--------------+
| 26MCI10246 |          47.00 |      94.00 | A+    | PASS         |
| 26MCI10247 |          41.00 |      82.00 | A     | PASS         |
| 26MCI10250 |          29.00 |      58.00 | C     | PASS         |
+------------+----------------+------------+-------+--------------+
+-------------------------------------------------+
| step                                            |
+---------------------------------+
| step                            |
+---------------------------------+
|  > 80 marks into a 50-mark paper |
+---------------------------------+
-------------
INSERT INTO results (student_id, exam_id, marks_obtained) VALUES (3, @demo_exam, 80)
-------------
```

```
ERROR 1644 (45000) at line 51: Result rejected: marks cannot exceed the maximum for this paper.
+-------------------------------------------------+
| step                                            |
+-------------------------------------------------+
|  > a second result for student 1 in the same exam |
+-------------------------------------------------+
-------------
INSERT INTO results (student_id, exam_id, marks_obtained) VALUES (1, @demo_exam, 40)
-------------

ERROR 1062 (23000) at line 54: Duplicate entry '1-5' for key 'uq_result_once'
+-------------------------------------------------+
| step                                            |
+-------------------------------------------------+
|  > a result for student 9999, who does not exist |
+-------------------------------------------------+
-------------
INSERT INTO results (student_id, exam_id, marks_obtained) VALUES (9999, @demo_exam, 40)
-------------

ERROR 1452 (23000) at line 57: Cannot add or update a child row: a foreign key constraint fails
(`result_system`.`results`, CONSTRAINT `fk_result_student` FOREIGN KEY (`student_id`) REFERENCES `students`
(`student_id`) ON DELETE CASCADE)
+-------------------+
| step              |
+-------------------+
|  > a negative mark |
+-------------------+
-------------
INSERT INTO results (student_id, exam_id, marks_obtained) VALUES (3, @demo_exam, -5)
-------------

ERROR 4025 (23000) at line 60: CONSTRAINT `chk_marks_positive` failed for `result_system`.`results`
+-----------------------------------------------+
| step                                          |
```

Output 7.3 Rank and class statistics

```
+---------+-----------+--------------+---------------+----------+-----------+-------+--------------+
| rank_no | roll_no   | full_name    | marks_obtained | max_marks | percentage | grade | result_status |
+---------+-----------+--------------+---------------+----------+-----------+-------+--------------+
|       1 | 26MCI10246 | Puneet Gupta  |         47.00 |       50 |      94.00 | A+    | PASS          |
|       2 | 26MCI10247 | Aditya Sharma |         41.00 |       50 |      82.00 | A     | PASS          |
|       3 | 26MCI10250 | Simran Bedi   |         29.00 |       50 |      58.00 | C     | PASS          |
+---------+-----------+--------------+---------------+----------+-----------+-------+--------------+
+-----------------------------------------------+
| step                                          |
```

Output 7.4 A corrected mark is regraded, and the audit trail

```
+-----------+-----------+-----------+
| result_id | old_marks | new_marks |
+-----------+-----------+-----------+
|        27 |        41 |        19 |
+-----------+-----------+-----------+
+---------------+------------+-------+--------------+
| marks_obtained | percentage | grade | result_status |
+---------------+------------+-------+--------------+
|          19.00 |      38.00 | F     | FAIL         |
+---------------+------------+-------+--------------+
+-----------------------------------------------+
| step                                          |
+-----------+--------+-----------+-----------+------------+
| result_id | action | old_marks | new_marks | changed_by |
+-----------+--------+-----------+-----------+------------+
|        26 | INSERT |      NULL |     47.00 | demo       |
|        27 | INSERT |      NULL |     41.00 | demo       |
|        28 | INSERT |      NULL |     29.00 | demo       |
|        27 | UPDATE |     41.00 |     19.00 | demo       |
+-----------+--------+-----------+-----------+------------+
+---------------------------------------+
| step                                  |
```

The same behaviour through the web interface is shown in the figures below. The first is the dashboard, where a mark is entered. The second is the page that runs six deliberately broken statements and reports the database's own error message for each. The third is the analytics page, every figure of which is a SQL aggregate.

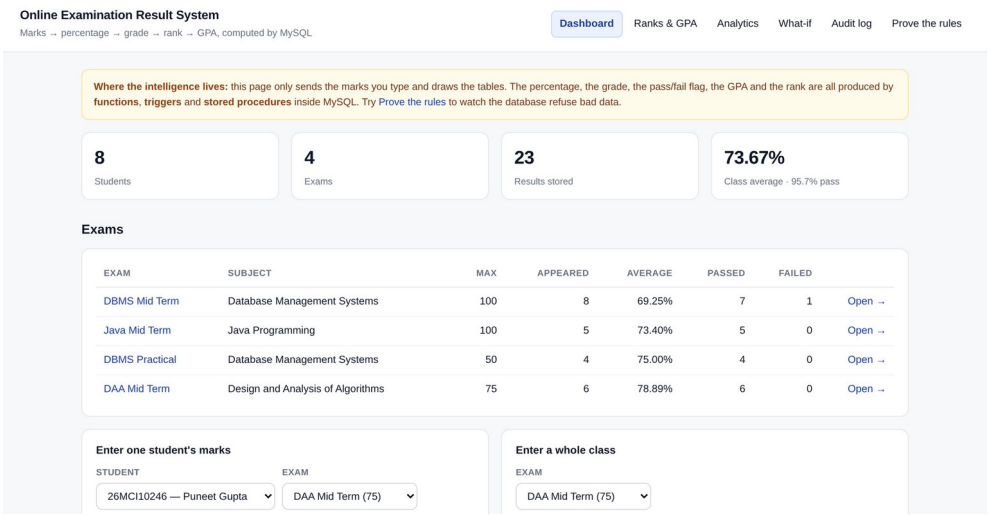


Fig. 2. The dashboard. The marks entry forms are at the centre; the leaderboard below shows the GPA and the overall rank from v_student_totals.

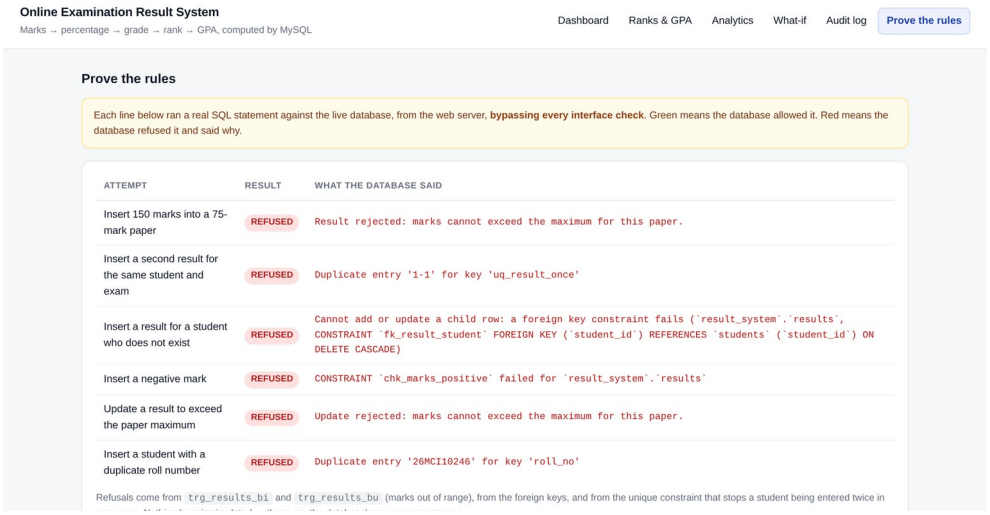


Fig. 3. The page that proves the rules. Each line runs a real statement from the web server, bypassing the interface, and prints what the database said.

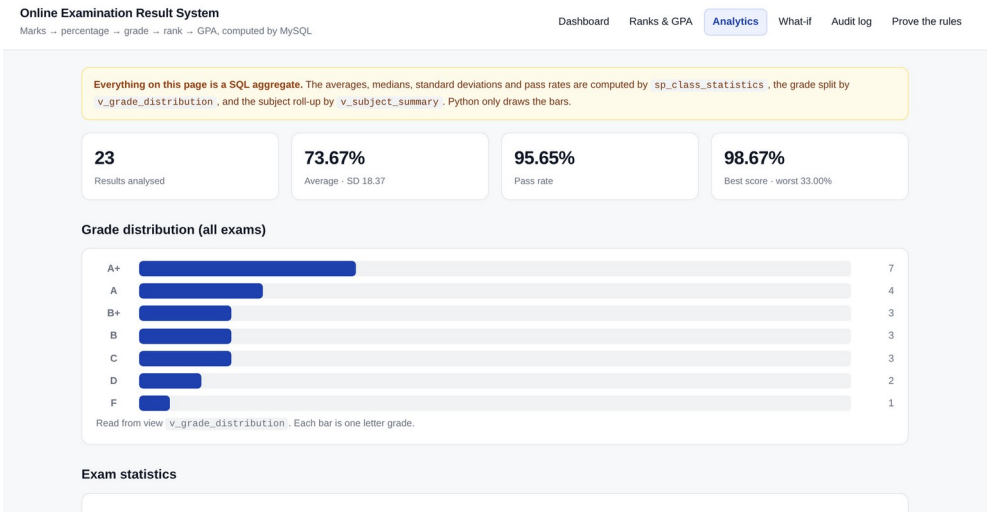


Fig. 4. The analytics page. The mean, median, standard deviation and pass rate are computed by sp_class_statistics.

8. Functionalities

Teacher

- **Generate a result:** Entering a mark for a student in an exam creates the row and fills the percentage, the grade and the status. A second entry for the same student in the same exam is refused.
- **Enter a whole class:** A list of students and marks is submitted in one call. Rows that duplicate an existing result, or that exceed the maximum, are counted and skipped.
- **Correct a mark:** Changing a mark updates the row; the before-update trigger recalculates the percentage and the grade, and the audit trigger records the old and the new value.
- **Delete a result:** Deleting removes the row and leaves a record of the deletion in the audit trail.
- **View the merit list:** The students of one exam are listed in rank order, with marks, percentage, grade and status.
- **View analytics:** Average, median, standard deviation, highest, lowest, pass rate and topper for each exam, and a roll-up by subject.
- **Print a mark sheet:** One student's results for every exam, with totals, overall percentage, GPA and overall rank.

Student

- **See own record:** The student report lists each exam taken, the marks, the percentage, the grade, the status and the rank within that exam.
- **See overall standing:** The total marks, overall percentage, GPA and overall rank in the class.
- **Check a possible score:** The what-if page answers the question of what a given mark would be worth, using the same functions, without storing anything.

Automatic

- **Rule enforcement:** A mark above the paper's maximum, a negative mark, a duplicate result and a result for a student who does not exist are all refused by the database, whichever client attempts them.
- **Derived values:** Percentage, grade and pass or fail follow the mark with no code outside the database involved.
- **Audit trail:** Every insert, update and delete is recorded with the old value, the new value, the user and the time.
- **Live ranking:** Ranks and statistics are recalculated on each read, so a corrected mark immediately changes the rank of everyone in that exam.

9. Conclusion

The project moved the rules of the result system out of the web page and into the database. The consequence is visible in the code: the Flask route that enters a result is a dozen lines that validate a form and call a procedure, and it contains no arithmetic. The same rules apply when a statement is issued from the mysql console, which is what the proof page demonstrates. Adding a second client, or replacing the web interface altogether, would not require the rules to be rewritten.

The reporting work became shorter as well. A merit list is a view with a window function, a class average is an aggregate in a procedure, and a student's GPA is an average over a function that maps a grade to points. Because none of these are stored, a corrected mark needs no rebuilding of totals; the next read is already correct. The audit trail, written by triggers rather than by the

application, recorded changes that were made outside the application during testing, which an application-level log would have missed.

10. Future Work

- Add authentication and separate the teacher's rights from the student's, so that a student can read only their own results.
- Grade the answer papers question by question instead of storing one total for an exam, and let the trigger recompute the total from the parts.
- Add a scheduled event that closes an exam automatically at its deadline, using the MySQL event scheduler.
- Export a mark sheet as PDF on the server instead of relying on the browser's print dialog.
- Add a moderation pass, in which a change to the grade scale is applied to existing results inside one transaction and recorded in the audit trail.

11. Learning Outcomes

- Writing a before-insert trigger that both validates and computes, and knowing why the derived columns must be assigned to NEW rather than returned.
- Using SIGNAL to raise an application error from inside a trigger, and a handler in the calling procedure to turn it into a message for the user.
- Thinking about data types: a percentage computed in a narrow DECIMAL overflows before a clamp is applied, which is a bug that only appears on input that should never arrive.
- Using an explicit cursor with a continue handler so that one bad row in a batch is skipped instead of stopping the whole batch.
- Understanding that a window function gives a rank without storing it, and why that is preferable to a column that has to be maintained.
- Testing through the interface and not only through the console: an error in the batch entry existed in the web layer while the procedure underneath was correct.

12. GitHub Link

[ADD YOUR REPOSITORY LINK HERE]

13. Bibliography

[1] Oracle Corporation. MySQL 8.0 Reference Manual: Stored Objects. https://dev.mysql.com/doc/refman/8.0/en/stored-objects.html

[2] Oracle Corporation. MySQL 8.0 Reference Manual: Window Functions. https://dev.mysql.com/doc/refman/8.0/en/window-functions.html

[3] MariaDB Foundation. MariaDB Documentation: Triggers and Stored Procedures. https://mariadb.com/kb/en/

[4] Pallets Projects. Flask Documentation. https://flask.palletsprojects.com/

[5] PyMySQL. PyMySQL Documentation. https://pymysql.readthedocs.io/

[6] Elmasri, R. and Navathe, S. B. Fundamentals of Database Systems, 7th edition. Pearson, 2016.

[7] Silberschatz, A., Korth, H. F. and Sudarshan, S. Database System Concepts, 7th edition. McGraw-Hill, 2019.